

-----Progetto gestione esami-----

Ho sviluppato il frontend insieme ad Aglo Alushaj, in particolare ho lavorato sulla Architettura MVP(model-view-presenter).

L'obiettivo di questa architettura è rendere il codice più manutenibile, facilitare il testing e separare la logica business, interfaccia grafica e logica di coordinamento.

Ho sviluppato 3 classi fondamentali: ExamModel, ExamView e ExamPresenter.

--ExamModel--

Gestisce la logica Business e i dati. Nel nostro progetto rappresenta il cuore logico dell'applicazione:

- Contiene i dati dei esami.

- gestisce le operazioni della CRUD(Create – Read – Delete) attraverso le rispettive funzioni : add(esame), getAll(), remove(id).

- Calcola le statistiche (media aritmetica, media ponderata, proiezione voto di laurea).

Attualmente ExamModel utilizza dati Mock, simulando un backend reale.

--ExamView--

Si occupa principalmente della proiezione grafica.
Nel progetto:

- Disegna la tabella degli esami.
- Aggiorna i valori delle statistiche.
- Intercetta le azioni del utente , per esempio il “click”
- Non conosce il model.
- Non contiene la logica business
- Espone solo metodi come : renderTable, updateStats, bindAddExam.

--ExamPresenter--

è il componente centrale del'MVP, il suo ruolo è:

- Ricevere eventi dalla View.
- Chiedere i dati al Model.
- Aggiornare la View con i risultati.

Nel progetto il Presenter:

- Inizializza l'applicazione.
- Recupera la lista dei esami dal Model.
- Aggiorna la tabella e le statistiche .
- Gestisce l'aggiunta e la eliminazione dei esami

--Flusso di funzionamento dell'applicazione--

1. L'utente apre la pagina
2. il main.js crea:
 - ExamModel
 - ExamView
 - ExamPresenter
3. Il Presenter inizializza l'applicazione
4. Il Model fornisce i dati
5. La View visualizza quest'ultimi (i dati)
6. Ogni iterazione passa dal Presenter

Questo garantisce un flusso ordinato e prevedibile.

--Conclusione--

L'adozione dell'architettura MVP ha permesso di strutturare il progetto in modo ordinato e scalabile, rendendo chiaro il ruolo di ogni componente.

Anche in assenza del backend definitivo, l'uso di dati Mock ha consentito di sviluppare e testare il frontend in modo realistico.